

TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue ETL pipeline. It defines the technical architecture, data flows, transformation logic, and runtime configurations necessary to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytical aggregates.

Scope

- In Scope:
 - AWS Glue PySpark job implementation specifications
 - S3 source and target path configurations
 - AWS Glue Data Catalog table definitions and registration logic
 - Data quality transformation logic (null removal, deduplication)
 - Apache Hudi table format configuration for SCD Type 2 implementation
 - Incremental processing logic using Hudi upsert capabilities
 - Customer aggregate spend calculation and storage
 - Error handling, logging, and validation specifications
 - Runtime parameter definitions
- Out of Scope:
 - Infrastructure provisioning (IAM roles, S3 buckets, Glue database creation)
 - CI/CD pipeline configuration
 - Monitoring dashboard setup
 - Data encryption implementation
 - Performance tuning beyond standard Spark configurations
 - Unit test implementation
 - Data retention automation

Assumptions

1. AWS Glue database `gen\_ai\_poc\_databrickscoe` exists prior to job execution

2. S3 paths are accessible with appropriate IAM permissions attached to Glue job role
3. CSV files use comma delimiter with header row
4. Data types for all columns will be inferred from CSV or explicitly cast during processing
5. "Null" string literal and actual NULL values are the only null representations
6. Duplicate detection uses all columns with exact match logic
7. SCD Type 2 change detection compares all business columns (excluding SCD metadata columns)
8. Date fields in order data are in string format and will be cast to date type during processing
9. Hudi record key for ordersummary will be a composite of OrderId and CustId
10. Hudi precombine key will be OpTs (operation timestamp)
11. Incremental processing will use full comparison approach (no CDC available)
12. Customer baseline for incremental processing will be stored in S3 as checkpoint
13. PricePerUnit and Qty are numeric types suitable for multiplication
14. Glue job will run with Glue 4.0 runtime with Hudi libraries pre-installed
15. Target S3 paths will be created if they do not exist
16. Parquet format will be used for non-Hudi catalog tables
17. No partitioning strategy is defined; tables will be non-partitioned unless specified
18. Overwrite mode will be used for catalog table writes except for Hudi tables (upsert mode)
-

2. Technical Requirements

TR-INGEST-001: Technical Implementation for Customer Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 source path and load into Spark DataFrame with schema inference and validation.

- Source Details:
- Source Type: S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- File Format: CSV
- Delimiter: Comma (,)
- Header: Yes (first row)
- Encoding: UTF-8 (assumed)
- Target Details:

- Target Type: In-memory Spark DataFrame
- Target Name: df_customer_raw
- Input Schema:

	Column Name	Data Type	Nullable	Notes	
	-----	-----	-----	-----	
	CustId	String	Yes	Primary key candidate	
	Name	String	Yes	Customer name	
	EmailId	String	Yes	Email address	
	Region	String	Yes	Geographic region	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Use Spark ``spark.read.format("csv")`` with options:

- ``header=true``
- ``inferSchema=true``
- ``mode=FAILFAST``

2. Read all CSV files from source path using wildcard pattern

3. Load into DataFrame ``df_customer_raw``

4. Perform initial row count and log to CloudWatch

5. Validate that expected columns exist in DataFrame schema

- Validation Rules:

- Source path must exist and contain at least one .csv file
- CSV must contain header row with columns: CustId, Name, EmailId, Region
- File must be parseable as valid CSV format
- Minimum row count: 0 (empty file is valid but logged as warning)

- Error Handling and Logging:

- Error Condition: Source path does not exist

- Action: Raise exception with message "Source path not found:
s3://adif-sdlc/sdlc_wizard/customerdata/"

- Log Level: ERROR

- Error Condition: CSV parsing failure

- Action: Raise exception with message "Failed to parse CSV files in source path"

- Log Level: ERROR

- Error Condition: Schema mismatch (missing expected columns)

- Action: Raise exception with message "Schema validation failed. Expected columns: CustId, Name, EmailId, Region"
- Log Level: ERROR
- Success Condition: Log row count with message "Successfully loaded {count} customer records from source"
- Log Level: INFO
- Runtime Parameters:
 - `SOURCE\_CUSTOMER\_PATH`: s3://adif-sdlc/sdlc_wizard/customerdata/
 - `CSV\_DELIMITER`: ","
 - `CSV\_HEADER`: "true"
 - `INFER\_SCHEMA`: "true"
- Operational Notes:
 - Use FAILFAST mode to ensure immediate failure on malformed data
 - Consider using schema enforcement in production to avoid schema drift
-

TR-INGEST-002: Technical Implementation for Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order CSV files from S3 source path and load into Spark DataFrame with schema inference and validation.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
 - File Format: CSV
 - Delimiter: Comma (,)
 - Header: Yes (first row)
 - Encoding: UTF-8 (assumed)
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: df_order_raw
- Input Schema:

	Column Name	Data Type	Nullable	Notes	
--	-------------	-----------	----------	-------	--

	-----	-----	-----	-----	
	OrderId	String	Yes	Primary key candidate	
	ItemName	String	Yes	Product/item name	
	PricePerUnit	String	Yes	Will be cast to Decimal	
	Qty	String	Yes	Will be cast to Integer	
	Date	String	Yes	Will be cast to Date	
	CustId	String	Yes	Foreign key to Customer	

- Output Schema:

Same as Input Schema (in-memory DataFrame with string types initially)

- Processing / Transformation Logic:

1. Use Spark ``spark.read.format("csv")`` with options:

- ``header=true``
- ``inferSchema=true``
- ``mode=FAILFAST``

2. Read all CSV files from source path using wildcard pattern

3. Load into DataFrame ``df_order_raw``

4. Perform initial row count and log to CloudWatch

5. Validate that expected columns exist in DataFrame schema

- Validation Rules:

- Source path must exist and contain at least one .csv file
- CSV must contain header row with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- File must be parseable as valid CSV format
- Minimum row count: 0 (empty file is valid but logged as warning)

- Error Handling and Logging:

- Error Condition: Source path does not exist

• Action: Raise exception with message "Source path not found:
s3://adif-sdlc/sdlc_wizard/orderdata/"

- Log Level: ERROR

- Error Condition: CSV parsing failure

• Action: Raise exception with message "Failed to parse CSV files in source path"

- Log Level: ERROR

- Error Condition: Schema mismatch (missing expected columns)

• Action: Raise exception with message "Schema validation failed. Expected columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId"

- Log Level: ERROR
- Success Condition: Log row count with message "Successfully loaded {count} order records from source"
- Log Level: INFO
- Runtime Parameters:
 - `SOURCE\_ORDER\_PATH`: s3://adif-sdlc/sdlc_wizard/orderdata/
 - `CSV\_DELIMITER`: ","
 - `CSV\_HEADER`: "true"
 - `INFER\_SCHEMA`: "true"
- Operational Notes:
 - Use FAILFAST mode to ensure immediate failure on malformed data
 - Type casting for numeric and date fields will occur in subsequent transformation steps
-

TR-CLEAN-001: Technical Implementation for Null Value Removal

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove all records containing string literal "Null" or actual NULL values from customer and order DataFrames.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name (Customer): df_customer_raw (from TR-INGEST-001)
 - Source Name (Order): df_order_raw (from TR-INGEST-002)
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name (Customer): df_customer_null_removed
 - Target Name (Order): df_order_null_removed
- Input Schema:
 - Customer: CustId, Name, EmailId, Region (all String type)
 - Order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId (all String type)
- Output Schema:

Same as Input Schema (subset of rows)

- Processing / Transformation Logic:

1. For Customer DataFrame:

- Filter out rows where any column equals string "Null" (case-sensitive)
- Filter out rows where any column is NULL (using `isNull()` check)

id").isNull()	col("Name").isNull()	col("EmailId").isNull()	col("Region").isNull()	(col("CustId") == "Null")	(col("Name") == "Null")	(col("EmailId") == "Null")
---------------	----------------------	-------------------------	------------------------	---------------------------	-------------------------	----------------------------

- Store result in `df\_customer\_null\_removed`
- Calculate and log removed row count: `removed\_count = df\_customer\_raw.count() - df\_customer\_null\_removed.count()`

2. For Order DataFrame:

- Filter out rows where any column equals string "Null" (case-sensitive)
- Filter out rows where any column is NULL (using `isNull()` check)

id").isNull()	col("Qty").isNull()	col("Date").isNull()	col("CustId").isNull()	(col("OrderId") == "Null")	(col("ItemName") == "Null")	(col("Qty") == "Null")
---------------	---------------------	----------------------	------------------------	----------------------------	-----------------------------	------------------------

- Store result in `df\_order\_null\_removed`
- Calculate and log removed row count: `removed\_count = df\_order\_raw.count() - df\_order\_null\_removed.count()`

• Validation Rules:

- All columns must be checked for both NULL and "Null" string
- Resulting DataFrame must not contain any NULL or "Null" values in any column
- Row count after filtering must be ≥ 0

• Error Handling and Logging:

- Warning Condition: All customer records removed
- Action: Log warning "All customer records contained null values and were removed"
- Log Level: WARN
- Continue Processing: Yes (with empty DataFrame)
- Warning Condition: All order records removed
- Action: Log warning "All order records contained null values and were removed"
- Log Level: WARN
- Continue Processing: Yes (with empty DataFrame)
- Success Condition: Log info message "Removed {count} customer records with null values. Remaining: {remaining}"
- Log Level: INFO
- Success Condition: Log info message "Removed {count} order records with null values. Remaining: {remaining}"
- Log Level: INFO

- Runtime Parameters:
- ``NULL_STRING_LITERAL``: "Null"
- Operational Notes:
- Consider using ``dropna()`` with ``how='any'`` for NULL removal, then apply string filter for "Null" literal
- Performance optimization: Use broadcast join if filtering logic becomes complex
-

TR-CLEAN-002: Technical Implementation for Duplicate Removal

- Related Functional Requirement ID(s): FR-CLEAN-002
- Objective:

Remove duplicate records from customer and order DataFrames based on all column values, retaining first occurrence.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name (Customer): `df_customer_null_removed` (from TR-CLEAN-001)
- Source Name (Order): `df_order_null_removed` (from TR-CLEAN-001)
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name (Customer): `df_customer_clean`
- Target Name (Order): `df_order_clean`
- Input Schema:
- Customer: CustId, Name, EmailId, Region (all String type)
- Order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId (all String type)
- Output Schema:

Same as Input Schema (subset of rows)

- Processing / Transformation Logic:
- 1. For Customer DataFrame:
- Apply ``dropDuplicates()`` on all columns: CustId, Name, EmailId, Region
- Implementation: ``df_customer_clean = df_customer_null_removed.dropDuplicates(["CustId", "Name", "EmailId", "Region"])``
- Calculate and log removed duplicate count: ``duplicate_count = df_customer_null_removed.count() - df_customer_clean.count()``

2. For Order DataFrame:

- Apply `dropDuplicates()` on all columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Implementation: `df_order_clean = df_order_null_removed.dropDuplicates(["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"])`
- Calculate and log removed duplicate count: `duplicate_count = df_order_null_removed.count() - df_order_clean.count()`

• Validation Rules:

- Deduplication must consider all columns in each dataset
- First occurrence of duplicate set must be retained (Spark default behavior)
- Resulting DataFrame must contain only unique records
- Row count after deduplication must be ≥ 0

• Error Handling and Logging:

- Warning Condition: All customer records except one are duplicates
- Action: Log warning "Only 1 unique customer record found after deduplication. Removed {count} duplicates"
- Log Level: WARN
- Continue Processing: Yes
- Warning Condition: All order records except one are duplicates
- Action: Log warning "Only 1 unique order record found after deduplication. Removed {count} duplicates"
- Log Level: WARN
- Continue Processing: Yes
- Success Condition: Log info message "Removed {count} duplicate customer records. Unique records: {remaining}"
- Log Level: INFO
- Success Condition: Log info message "Removed {count} duplicate order records. Unique records: {remaining}"
- Log Level: INFO
- Runtime Parameters:

None

• Operational Notes:

- `dropDuplicates()` uses Spark's default behavior of keeping first occurrence
- For large datasets, consider repartitioning before deduplication for performance
- Deduplication is performed in-memory; ensure sufficient executor memory

•

TR-CATALOG-001: Technical Implementation for Customer Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer data to S3 in Parquet format and register as a table in AWS Glue Data Catalog for Athena querying.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_customer_clean (from TR-CLEAN-002)
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_customer
- Target S3 Path: s3:///catalog/sdlc_wizard/customer/
- File Format: Parquet
- Compression: Snappy (default)
- Write Mode: Overwrite
- Input Schema:

	Column Name	Data Type	Nullable	
	-----	-----	-----	
	CustId	String	No	
	Name	String	No	
	EmailId	String	No	
	Region	String	No	

- Output Schema:

Same as Input Schema

- Processing / Transformation Logic:

1. Write DataFrame to S3 using Parquet format:

```
`df_customer_clean.write.mode("overwrite").format("parquet").save("s3:///catalog/sdlc_wizard/customer/")`
```

2. Create or update Glue Catalog table using Spark SQL:

```
```sql
```

```
CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard_customer
```

USING parquet

LOCATION 's3:///catalog/sdlc\_wizard/customer/'

```

3. Alternatively, use `df_customer_clean.write.mode("overwrite").format("parquet").saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard_customer")``

4. Log successful registration with row count

- Validation Rules:

- Target database ``gen_ai_poc_databrickscoe`` must exist
- S3 target path must be writable
- Parquet files must be created successfully
- Glue Catalog table must be queryable via Athena after registration
- Table schema in catalog must match DataFrame schema

- Error Handling and Logging:

- Error Condition: Database does not exist
- Action: Raise exception with message "Glue database not found: gen_ai_poc_databrickscoe"
- Log Level: ERROR
- Error Condition: S3 write failure
- Action: Raise exception with message "Failed to write customer data to S3: {error_details}"
- Log Level: ERROR
- Error Condition: Catalog registration failure
- Action: Raise exception with message "Failed to register table in Glue Catalog: {error_details}"
- Log Level: ERROR
- Success Condition: Log info message "Successfully registered customer table with {count} records at s3:///catalog/sdlc_wizard/customer/"
- Log Level: INFO

- Runtime Parameters:

- ``CATALOG_DATABASE``: gen_ai_poc_databrickscoe
- ``CATALOG_CUSTOMER_TABLE``: sdlc_wizard_customer
- ``CATALOG_CUSTOMER_PATH``: s3:///catalog/sdlc_wizard/customer/
- ``PARQUET_COMPRESSION``: snappy

- Operational Notes:

- Use `saveAsTable()` for automatic catalog registration
- Overwrite mode will replace existing data and update catalog metadata
- No partitioning is applied; table is non-partitioned

- Ensure Glue job has `glue:CreateTable` and `glue:UpdateTable` permissions
-

TR-CATALOG-002: Technical Implementation for Order Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned order data to S3 in Parquet format and register as a table in AWS Glue Data Catalog for Athena querying.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_order_clean (from TR-CLEAN-002)
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: sdlc_wizard_order
 - Target S3 Path: s3:///catalog/sdlc_wizard/order/
 - File Format: Parquet
 - Compression: Snappy (default)
 - Write Mode: Overwrite
- Input Schema:

| | Column Name | Data Type | Nullable | |
|--|--------------|-----------|----------|--|
| | ----- | ----- | ----- | |
| | OrderId | String | No | |
| | ItemName | String | No | |
| | PricePerUnit | String | No | |
| | Qty | String | No | |
| | Date | String | No | |
| | CustId | String | No | |

- Output Schema:

Same as Input Schema

- Processing / Transformation Logic:
 1. Write DataFrame to S3 using Parquet format:

- ``df_order_clean.write.mode("overwrite").format("parquet").save("s3:///catalog/sdlc_wizard/order/"`)`

2. Create or update Glue Catalog table using Spark SQL:

```
```sql
```

```
CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard_order
USING parquet
LOCATION 's3:///catalog/sdlc_wizard/order/'
```

```
```
```

3. Alternatively, use ``df_order_clean.write.mode("overwrite").format("parquet").saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard_order")``

4. Log successful registration with row count

- Validation Rules:

- Target database ``gen_ai_poc_databrickscoe`` must exist
- S3 target path must be writable
- Parquet files must be created successfully
- Glue Catalog table must be queryable via Athena after registration
- Table schema in catalog must match DataFrame schema

- Error Handling and Logging:

- Error Condition: Database does not exist
- Action: Raise exception with message "Glue database not found: `gen_ai_poc_databrickscoe`"
- Log Level: ERROR
- Error Condition: S3 write failure
- Action: Raise exception with message "Failed to write order data to S3: {error_details}"
- Log Level: ERROR
- Error Condition: Catalog registration failure
- Action: Raise exception with message "Failed to register table in Glue Catalog: {error_details}"
- Log Level: ERROR
- Success Condition: Log info message "Successfully registered order table with {count} records at `s3:///catalog/sdlc_wizard/order/`"
- Log Level: INFO

- Runtime Parameters:

- ``CATALOG_DATABASE``: `gen_ai_poc_databrickscoe`
- ``CATALOG_ORDER_TABLE``: `sdlc_wizard_order`
- ``CATALOG_ORDER_PATH``: `s3:///catalog/sdlc_wizard/order/`

- `PARQUET\_COMPRESSION`: snappy
- Operational Notes:
 - Use `saveAsTable()` for automatic catalog registration
 - Overwrite mode will replace existing data and update catalog metadata
 - No partitioning is applied; table is non-partitioned
 - Ensure Glue job has `glue:CreateTable` and `glue:UpdateTable` permissions
-

TR-SCD2-001: Technical Implementation for SCD Type 2 Order Summary

- Related Functional Requirement ID(s): FR-SCD2-001
- Objective:

Join customer and order data, implement SCD Type 2 logic to track historical changes, and write to S3 using Apache Hudi format with Glue Catalog registration.

- Source Details:
 - Source Type (Customer): In-memory Spark DataFrame
 - Source Name (Customer): df_customer_clean (from TR-CLEAN-002)
 - Source Type (Order): In-memory Spark DataFrame
 - Source Name (Order): df_order_clean (from TR-CLEAN-002)
 - Source Type (Existing Target): Hudi Table on S3
 - Source Path (Existing Target): s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
 - Target Type: Apache Hudi Table (Copy-on-Write)
 - Target Database: gen_ai_poc_databricksco
 - Target Table: ordersummary
 - Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Hudi Table Type: COPY_ON_WRITE
 - Hudi Record Key: Composite key (OrderId + CustId)
 - Hudi Precombine Key: OpTs
 - Hudi Operation: UPSERT
 - Write Mode: Append (Hudi manages versioning)
- Input Schema (Joined Data):

| | Column Name | Data Type | Source | Notes | |
|--|-------------|-----------|--------|-------|--|
| | ----- | ----- | ----- | ----- | |

| | | | |
|--------------|---------------|----------|---|
| OrderId | String | Order | Direct map from df_order_clean.OrderId |
| ItemName | String | Order | Direct map from df_order_clean.ItemName |
| PricePerUnit | Decimal(10,2) | Order | Cast from df_order_clean.PricePerUnit |
| Qty | Integer | Order | Cast from df_order_clean.Qty |
| Date | Date | Order | Cast from df_order_clean.Date |
| CustId | String | Both | Join key |
| Name | String | Customer | Direct map from df_customer_clean.Name |
| EmailId | String | Customer | Direct map from df_customer_clean.EmailId |
| Region | String | Customer | Direct map from df_customer_clean.Region |

• Output Schema (SCD Type 2 with Metadata):

| Column Name | Data Type | Nullable | Notes |
|--------------|---------------|----------|---|
| ----- | ----- | ----- | ----- |
| OrderId | String | No | Business key part 1 |
| CustId | String | No | Business key part 2 |
| ItemName | String | No | Business attribute |
| PricePerUnit | Decimal(10,2) | No | Business attribute |
| Qty | Integer | No | Business attribute |
| Date | Date | No | Business attribute |
| Name | String | No | Customer attribute |
| EmailId | String | No | Customer attribute |
| Region | String | No | Customer attribute |
| IsActive | Boolean | No | SCD Type 2 flag (true=current) |
| StartDate | Timestamp | No | SCD Type 2 effective start |
| EndDate | Timestamp | Yes | SCD Type 2 effective end (null=current) |
| OpTs | Timestamp | No | Operation timestamp (precombine key) |

• Processing / Transformation Logic:

1. Join Customer and Order Data:

```
```python
df_joined = df_order_clean.join(df_customer_clean, on="CustId", how="inner")
```
```

2. Type Casting:

```
```python
df_joined = df_joined.withColumn("PricePerUnit", col("PricePerUnit").cast("decimal(10,2)"))
df_joined = df_joined.withColumn("Qty", col("Qty").cast("int"))
df_joined = df_joined.withColumn("Date", col("Date").cast("date"))
```
```

...

3. Add SCD Type 2 Metadata Columns:

```
```python
from pyspark.sql.functions import current_timestamp, lit

df_new = df_joined.withColumn("IsActive", lit(True))
df_new = df_new.withColumn("StartDate", current_timestamp())
df_new = df_new.withColumn("EndDate", lit(None).cast("timestamp"))
df_new = df_new.withColumn("OpTs", current_timestamp())
```
```

4. Read Existing Hudi Table (if exists):

```
```python
try:
df_existing = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
except:
df_existing = None # First run, no existing data
```
```

5. SCD Type 2 Logic:

- If `df\_existing` is None (first run):
- Write `df\_new` directly to Hudi table
- If `df\_existing` exists:
- Create composite key: `composite\_key = concat(col("OrderId"), lit("\_"), col("CustId"))`
- Compare `df\_new` with active records in `df\_existing` (where IsActive=true)
- For each record in `df\_new`:
- New Record: If composite_key not in df_existing, insert with IsActive=true, EndDate=null
- Changed Record: If composite_key exists but business columns differ:
- Update existing record: set IsActive=false, EndDate=current_timestamp, OpTs=current_timestamp
- Insert new record: set IsActive=true, StartDate=current_timestamp, EndDate=null, OpTs=current_timestamp
- Unchanged Record: If composite_key exists and all business columns match, no action

6. Hudi Write Configuration:

```
```python
hudi_options = {
'hoodie.table.name': 'ordersummary',
```



```

'hoodie.datasource.write.recordkey.field': 'OrderId,CustId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.hive_sync.enable': 'true',
'hoodie.datasource.hive_sync.database': 'gen_ai_poc_databrickscoe',
'hoodie.datasource.hive_sync.table': 'ordersummary',
'hoodie.datasource.hive_sync.partition_extractor_class': 'org.apache.hudi.hive.NonPartitionedExtractor',
'hoodie.datasource.hive_sync.use_jdbc': 'false',
'hoodie.datasource.write.hive_style_partitioning': 'false'
}

```

```

df_final.write.format("hudi").options(hudi_options).mode("append").save("s3://adif-sdlc/curated/sdlc_w
izard/ordersummary/")

```

```

...

```

## 7. Catalog Registration:

- Hudi auto-sync will register/update table in Glue Catalog via `hive\_sync` options
- Validation Rules:
  - Join must be on CustId column present in both datasets
  - Composite key (OrderId + CustId) must be unique for active records (IsActive=true)
  - All active records must have EndDate=null
  - All inactive records must have EndDate populated
  - OpTs must be populated for all records
  - StartDate must be <= EndDate (when EndDate is not null)
  - Type casting must succeed for PricePerUnit, Qty, Date
- Error Handling and Logging:
  - Error Condition: Join produces no records
  - Action: Log warning "Join between customer and order data produced no records. Creating empty Hudi table."
  - Log Level: WARN
  - Continue Processing: Yes (create empty table structure)
  - Error Condition: Type casting failure
  - Action: Raise exception with message "Type casting failed for column {column\_name}: {error\_details}"
  - Log Level: ERROR

- Error Condition: Hudi write failure
- Action: Raise exception with message "Hudi write operation failed: {error\_details}"
- Log Level: ERROR
- Rollback: Hudi provides transactional writes; failed commits are rolled back automatically
- Error Condition: Multiple active records for same composite key after processing
- Action: Raise exception with message "Data integrity violation: Multiple active records found for key {composite\_key}"
- Log Level: ERROR
- Success Condition: Log info message "Successfully processed SCD Type 2 for ordersummary. New records: {new\_count}, Updated records: {update\_count}, Unchanged: {unchanged\_count}"
- Log Level: INFO
- Runtime Parameters:
  - `HUDI\_TABLE\_NAME`: ordersummary
  - `HUDI\_RECORD\_KEY`: OrderId,CustId
  - `HUDI\_PRECOMBINE\_KEY`: OpTs
  - `HUDI\_TABLE\_TYPE`: COPY\_ON\_WRITE
  - `HUDI\_OPERATION`: upsert
  - `HUDI\_TARGET\_PATH`: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
  - `HUDI\_DATABASE`: gen\_ai\_poc\_databrickscoe
  - `HUDI\_TABLE`: ordersummary
- Operational Notes:
  - Hudi COPY\_ON\_WRITE provides snapshot isolation and is optimized for read-heavy workloads
  - Precombine key (OpTs) determines which record to keep when duplicates exist during upsert
  - Hudi manages file versioning and compaction automatically
  - For large datasets, consider enabling Hudi async compaction
  - Ensure Glue job has Hudi libraries available (Glue 4.0 includes Hudi by default)
  - First run will create table structure; subsequent runs will perform upserts
- 

### ### TR-INCR-001: Technical Implementation for Incremental SCD2 Processing

- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Detect changes in customer data and trigger incremental SCD Type 2 updates to ordersummary table, processing only affected records.

- Source Details:
- Source Type (Current Customer): In-memory Spark DataFrame
- Source Name (Current Customer): df\_customer\_clean (from TR-CLEAN-002)
- Source Type (Previous Customer Baseline): Parquet files on S3
- Source Path (Previous Baseline): s3:///checkpoint/customer\_baseline/
- Source Type (Order): In-memory Spark DataFrame
- Source Name (Order): df\_order\_clean (from TR-CLEAN-002)
- Source Type (Existing Ordersummary): Hudi Table on S3
- Source Path (Existing Ordersummary): s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Target Details:
- Target Type: Apache Hudi Table (incremental update)
- Target Database: gen\_ai\_poc\_databricksco
- Target Table: ordersummary
- Target S3 Path: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Target Type (New Baseline): Parquet files on S3
- Target Path (New Baseline): s3:///checkpoint/customer\_baseline/
- Input Schema:
- Current Customer: CustId, Name, EmailId, Region (from df\_customer\_clean)
- Previous Customer Baseline: CustId, Name, EmailId, Region (from S3 checkpoint)
- Order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId (from df\_order\_clean)

- Output Schema:

Same as TR-SCD2-001 output schema (ordersummary with SCD Type 2 columns)

- Processing / Transformation Logic:

#### 1. Read Previous Customer Baseline:

```
```python
try:
df_customer_baseline = spark.read.format("parquet").load("s3:///checkpoint/customer_baseline/")
except:
df_customer_baseline = None # First run, no baseline exists
```
```

#### 2. Detect Changed Customers:

- If `df\_customer\_baseline` is None:

- All current customers are considered "changed" (first run)
- `df\_changed\_customers = df\_customer\_clean`
- If `df\_customer\_baseline` exists:
- Perform full outer join on CustId
- Identify new customers: present in current but not in baseline
- Identify updated customers: present in both but with different values in Name, EmailId, or Region
- Identify deleted customers: present in baseline but not in current (optional handling)

```
```python
```

```
df_comparison = df_customer_clean.alias("current").join(
df_customer_baseline.alias("baseline"),
on="CustId",
how="full_outer"
)
```

```
# New customers
```

```
df_new_customers = df_comparison.filter(col("baseline.CustId").isNull()).select("current.")
```

```
# Updated customers (any column changed)
```

```
df_updated_customers = df_comparison.filter(
(col("current.CustId").isNotNull()) &
(col("baseline.CustId").isNotNull()) &
(
```

(col("current.Name") != col("baseline.Name"))	
(col("current.EmailId") != col("baseline.EmailId"))	

```
(col("current.Region") != col("baseline.Region"))
```

```
)
```

```
).select("current.")
```

```
# Union new and updated
```

```
df_changed_customers = df_new_customers.union(df_updated_customers)
```

```
```
```

### 3. Filter Orders for Changed Customers:

```
```python
```

```
df_affected_orders = df_order_clean.join(df_changed_customers, on="CustId", how="inner")
```

```
```
```

#### 4. Apply SCD Type 2 Logic (Incremental):

- Join `df\_affected\_orders` with `df\_changed\_customers` to create joined dataset
- Read existing Hudi ordersummary table
- For each affected record:
- Apply same SCD Type 2 logic as TR-SCD2-001 but only for affected composite keys
- Use Hudi upsert to update existing records and insert new versions
- Implementation reuses TR-SCD2-001 logic but operates on subset of data

#### 5. Write Incremental Updates to Hudi:

```
```python
```

```
# Use same Hudi configuration as TR-SCD2-001
```

```
df_incremental_updates.write.format("hudi").options(hudi_options).mode("append").save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

```
```
```

#### 6. Update Customer Baseline:

```
```python
```

```
df_customer_clean.write.mode("overwrite").format("parquet").save("s3:///checkpoint/customer_baseline/")
```

```
```
```

- Validation Rules:
- Customer baseline must be readable if it exists
- Changed customer detection must identify at least 0 customers (empty is valid)
- Affected orders must be correctly filtered using changed customer CustIds
- SCD Type 2 logic must maintain data integrity (no duplicate active records)
- New baseline must be written successfully after processing
- Error Handling and Logging:
- Info Condition: No customer baseline exists (first run)
- Action: Log info "No customer baseline found. Processing all customers as changed (first run)."
- Log Level: INFO
- Info Condition: No customer changes detected
- Action: Log info "No customer changes detected. Skipping incremental processing."
- Log Level: INFO
- Continue Processing: Skip ordersummary update, update baseline, exit
- Error Condition: Baseline read failure (exists but unreadable)
- Action: Log error "Failed to read customer baseline. Falling back to full refresh."

- Log Level: ERROR
- Fallback: Process all customers (full refresh)
- Error Condition: Incremental Hudi update failure
- Action: Raise exception with message "Incremental Hudi update failed: {error\_details}"
- Log Level: ERROR
- Error Condition: Baseline write failure
- Action: Raise exception with message "Failed to update customer baseline: {error\_details}"
- Log Level: ERROR
- Success Condition: Log info message "Incremental processing complete. Changed customers: {count}, Affected orders: {order\_count}, Updated ordersummary records: {update\_count}"
- Log Level: INFO
- Runtime Parameters:
  - `CUSTOMER\_BASELINE\_PATH`: s3:///checkpoint/customer\_baseline/
  - `INCREMENTAL\_MODE`: true
  - All Hudi parameters from TR-SCD2-001
- Operational Notes:
  - Incremental processing significantly reduces runtime for large datasets
  - Customer baseline acts as checkpoint for change detection
  - If baseline is corrupted or missing, system falls back to full refresh
  - Hudi upsert operation is idempotent; reprocessing same data is safe
  - Consider implementing soft deletes for customers removed from source
  - Baseline update should occur only after successful ordersummary update
- 

### ### TR-AGG-001: Technical Implementation for Customer Aggregate Spend

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Join customer and order data, calculate total spending per customer per date, and write aggregated results to S3 with Glue Catalog registration.

- Source Details:
  - Source Type (Customer): In-memory Spark DataFrame
  - Source Name (Customer): df\_customer\_clean (from TR-CLEAN-002)
  - Source Type (Order): In-memory Spark DataFrame
  - Source Name (Order): df\_order\_clean (from TR-CLEAN-002)

- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen\_ai\_poc\_databricksco
- Target Table: customeraggregatespend
- Target S3 Path: s3://adif-sdlc/analytics/customeraggregatespend/
- File Format: Parquet
- Compression: Snappy (default)
- Write Mode: Overwrite
- Input Schema:
- Customer: CustId, Name, EmailId, Region
- Order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Output Schema:

|  | Column Name | Data Type     | Nullable | Derivation                                         |  |
|--|-------------|---------------|----------|----------------------------------------------------|--|
|  | -----       | -----         | -----    | -----                                              |  |
|  | Name        | String        | No       | Direct map from df_customer_clean.Name             |  |
|  | TotalAmount | Decimal(18,2) | No       | sum(PricePerUnit * Qty) grouped by Name, Date      |  |
|  | Date        | Date          | No       | Direct map from df_order_clean.Date (cast to date) |  |

- Processing / Transformation Logic:

#### 1. Join Customer and Order Data:

```
```python
df_joined = df_order_clean.join(df_customer_clean, on="CustId", how="inner")
```
```

#### 2. Type Casting:

```
```python
df_joined = df_joined.withColumn("PricePerUnit", col("PricePerUnit").cast("decimal(10,2)"))
df_joined = df_joined.withColumn("Qty", col("Qty").cast("int"))
df_joined = df_joined.withColumn("Date", col("Date").cast("date"))
```
```

#### 3. Calculate TotalAmount per Order:

```
```python
df_with_total = df_joined.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
```
```

#### 4. Aggregate by Customer Name and Date:

```
```python
from pyspark.sql.functions import sum as _sum

df_aggregated = df_with_total.groupBy("Name", "Date").agg(
    _sum("TotalAmount").alias("TotalAmount")
)
```
```

#### 5. Select Final Columns:

```
```python
df_final = df_aggregated.select("Name", "TotalAmount", "Date")
```
```

#### 6. Write to S3 and Register in Catalog:

```
```python
df_final.write.mode("overwrite").format("parquet").saveAsTable("gen_ai_poc_databrickscoe.customeragggregatespend")
```
```

- Alternatively, write to S3 first, then register:

```
```python
df_final.write.mode("overwrite").format("parquet").save("s3://adif-sdlc/analytics/customeragggregatespend/")

spark.sql("""
CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.customeragggregatespend
USING parquet
LOCATION 's3://adif-sdlc/analytics/customeragggregatespend/'
""")
```
```

#### • Validation Rules:

- Join must be on CustId column present in both datasets
- PricePerUnit and Qty must be numeric types suitable for multiplication
- TotalAmount calculation must not produce NULL values (unless input is NULL)
- Aggregation must group by both Name and Date
- Final output must contain exactly 3 columns: Name, TotalAmount, Date
- TotalAmount must be non-negative (log warning if negative values found)



- Error Handling and Logging:
- Warning Condition: Join produces no records
- Action: Log warning "Join between customer and order data produced no records. Creating empty aggregate table."
- Log Level: WARN
- Continue Processing: Yes (create empty table structure)
- Error Condition: Type casting failure
- Action: Raise exception with message "Type casting failed for column {column\_name}: {error\_details}"
- Log Level: ERROR
- Error Condition: Aggregation failure
- Action: Raise exception with message "Aggregation operation failed: {error\_details}"
- Log Level: ERROR
- Warning Condition: Negative TotalAmount values detected
- Action: Log warning "Negative TotalAmount values detected. Count: {negative\_count}. Review source data quality."
- Log Level: WARN
- Continue Processing: Yes
- Error Condition: S3 write failure
- Action: Raise exception with message "Failed to write aggregate data to S3: {error\_details}"
- Log Level: ERROR
- Error Condition: Catalog registration failure
- Action: Raise exception with message "Failed to register customeraggregatespend table in Glue Catalog: {error\_details}"
- Log Level: ERROR
- Success Condition: Log info message "Successfully created customeraggregatespend table with {count} aggregated records at s3://adif-sdlc/analytics/customeraggregatespend/"
- Log Level: INFO
- Runtime Parameters:
- `ANALYTICS\_DATABASE`: gen\_ai\_poc\_databricksco
- `ANALYTICS\_TABLE`: customeraggregatespend
- `ANALYTICS\_PATH`: s3://adif-sdlc/analytics/customeraggregatespend/
- `PARQUET\_COMPRESSION`: snappy
- Operational Notes:
- Overwrite mode ensures fresh aggregate calculation on each run

- Consider partitioning by Date for large datasets (not specified in FRD)
- TotalAmount precision (18,2) accommodates large aggregate values
- Aggregation is performed in-memory; ensure sufficient executor memory for large datasets
- Table is queryable via Athena immediately after successful write
- 

## ## 3. Runtime Strategy Notes

### ### Authoring Language

- Primary Language: PySpark (Python 3.9+)
- Rationale: PySpark provides native integration with AWS Glue, Spark DataFrame API, and Hudi libraries

### ### Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Glue Version: 4.0 (includes Spark 3.3.0, Python 3.10, Hudi 0.12.1)
- Worker Type: G.1X or G.2X (based on data volume)
- Number of Workers: Configurable (start with 2-5 workers, scale based on performance testing)
- Job Timeout: 2880 minutes (48 hours, adjust based on data volume)
- Max Concurrent Runs: 1 (to prevent concurrent SCD Type 2 conflicts)
- Job Bookmark: Disabled (using custom incremental logic via customer baseline)

### ### Glue Job Type Expectations

- Script Type: Python Shell or Spark (Spark recommended for data volume and Hudi support)
- Execution Mode: Standard (not Flex, to ensure consistent performance)
- Spark Configuration:
  - `spark.sql.adaptive.enabled`: true
  - `spark.sql.adaptive.coalescePartitions.enabled`: true
  - `spark.serializer`: org.apache.spark.serializer.KryoSerializer
  - `spark.sql.hive.convertMetastoreParquet`: false (for Hudi compatibility)
- Hudi Configuration: Embedded in code (see TR-SCD2-001)
- Logging: CloudWatch Logs enabled with continuous logging
- Metrics: CloudWatch Metrics enabled for job monitoring
- 

## ## 4. Data Design Details

### ### Source Paths / Source Systems

| Source Entity         | Source Type | Source Path                                      | Format  | Notes                                  |
|-----------------------|-------------|--------------------------------------------------|---------|----------------------------------------|
| -----                 | -----       | -----                                            | -----   | -----                                  |
| Customer              | S3 CSV      | s3://adif-sdlc/sdlc_wizard/customerdata/         | CSV     | Raw customer master data               |
| Order                 | S3 CSV      | s3://adif-sdlc/sdlc_wizard/orderdata/            | CSV     | Raw order transaction data             |
| Customer Baseline     | S3 Parquet  | s3://<BUCKET_NAME>/checkpoint/customer_baseline/ | Parquet | Checkpoint for incremental processing  |
| Existing Ordersummary | S3 Hudi     | s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ | Hudi    | Existing SCD Type 2 table (if present) |

### ### Target Paths / Target Tables

| Target Type         | Target Path                                      | Format     | Database                | Table Name      |
|---------------------|--------------------------------------------------|------------|-------------------------|-----------------|
| -----               | -----                                            | -----      | -----                   | -----           |
| Glue Catalog        | s3://<BUCKET_NAME>/catalog/sdlc_wizard/customer/ | Parquet    | gen_ai_poc_databricksco | sdlc_wizard_cus |
| Glue Catalog        | s3://<BUCKET_NAME>/catalog/sdlc_wizard/order/    | Parquet    | gen_ai_poc_databricksco | sdlc_wizard_ord |
| Glue Catalog (Hudi) | s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ | Hudi (COW) | gen_ai_poc_databricksco | ordersummary    |
| Glue Catalog        | s3://adif-sdlc/analytics/customeraggregatespend/ | Parquet    | gen_ai_poc_databricksco | customeraggrega |
| S3 Parquet          | s3://<BUCKET_NAME>/checkpoint/customer_baseline/ | Parquet    | N/A                     | N/A             |

### ### Catalog Registration Expectations

- All target tables must be registered in AWS Glue Data Catalog database: `gen\_ai\_poc\_databricksco`
- Catalog registration occurs automatically via `saveAsTable()` or Hudi `hive_sync`
- Tables must be queryable via Amazon Athena immediately after job completion
- Catalog metadata must reflect current schema and S3 location
- No manual catalog updates required; job handles all registration

### ### Partitioning Expectations

- Customer Catalog Table: Non-partitioned
- Order Catalog Table: Non-partitioned
- Ordersummary Hudi Table: Non-partitioned (Hudi manages internal partitioning)
- Customer Aggregate Spend Table: Non-partitioned
- Note: Partitioning strategy not specified in FRD; all tables are non-partitioned by default

### ### File Formats / Table Formats

| Entity                   | Format     | Compression | Notes                                              |  |
|--------------------------|------------|-------------|----------------------------------------------------|--|
| -----                    | -----      | -----       | -----                                              |  |
| Customer Catalog         | Parquet    | Snappy      | Columnar format for efficient querying             |  |
| Order Catalog            | Parquet    | Snappy      | Columnar format for efficient querying             |  |
| Ordersummary             | Hudi (COW) | Snappy      | Transactional table format with SCD Type 2 support |  |
| Customer Aggregate Spend | Parquet    | Snappy      | Columnar format for analytical queries             |  |
| Customer Baseline        | Parquet    | Snappy      | Checkpoint storage for incremental processing      |  |

### ### Update / Overwrite / Append Behavior

| Target Entity            | Write Mode      | Behavior                            | Rationale                                               |
|--------------------------|-----------------|-------------------------------------|---------------------------------------------------------|
| -----                    | -----           | -----                               | -----                                                   |
| Customer Catalog         | Overwrite       | Replace all data on each run        | Full refresh of cleaned customer data                   |
| Order Catalog            | Overwrite       | Replace all data on each run        | Full refresh of cleaned order data                      |
| Ordersummary             | Append (Upsert) | Hudi upsert based on record key     | SCD Type 2 requires versioning and history preservation |
| Customer Aggregate Spend | Overwrite       | Replace all data on each run        | Aggregate recalculated from source on each run          |
| Customer Baseline        | Overwrite       | Replace with current customer state | Checkpoint for next incremental run                     |

- Note: No staging tables, intermediate layers, or derived sources are introduced. All processing occurs in-memory using Spark DataFrames, with direct reads from original sources and direct writes to final targets.

- 

## ## 5. Processing Details

### ### Data Quality Rules

#### 1. Null Removal:

- Remove rows where any column contains string literal "Null" (case-sensitive)
- Remove rows where any column contains actual NULL value
- Apply to both customer and order datasets
- Log count of removed records

#### 2. Duplicate Removal:

- Identify duplicates based on all columns in each dataset
- Retain first occurrence of duplicate set
- Remove subsequent duplicates
- Apply to both customer and order datasets after null removal
- Log count of removed duplicates

#### 3. Type Validation:

- Validate that PricePerUnit can be cast to Decimal(10,2)
- Validate that Qty can be cast to Integer
- Validate that Date can be cast to Date type
- Fail job if type casting fails for any record

#### 4. Business Rule Validation:

- TotalAmount calculation must not produce NULL (unless input is NULL)
- Log warning if TotalAmount is negative (data quality issue)

- Composite key (OrderId + CustId) must be unique for active records in ordersummary

### ### Deduplication Rules

- Scope: Customer and Order datasets independently
- Comparison: All columns must match exactly for records to be considered duplicates
- Retention: First occurrence in DataFrame is retained
- Implementation: Spark `dropDuplicates()` function with all column names
- Timing: After null removal, before catalog registration and joins

### ### Join Rules

#### 1. Customer-Order Join (for SCD Type 2):

- Join Type: Inner join
- Join Key: CustId
- Left Dataset: df\_order\_clean
- Right Dataset: df\_customer\_clean
- Result: All orders with matching customer information
- Null Handling: No nulls expected due to prior cleaning; inner join excludes non-matches

#### 2. Customer-Order Join (for Aggregation):

- Join Type: Inner join
- Join Key: CustId
- Left Dataset: df\_order\_clean
- Right Dataset: df\_customer\_clean
- Result: All orders with matching customer information
- Null Handling: No nulls expected due to prior cleaning; inner join excludes non-matches

#### 3. Customer Baseline Comparison (for Incremental Processing):

- Join Type: Full outer join
- Join Key: CustId
- Left Dataset: df\_customer\_clean (current)
- Right Dataset: df\_customer\_baseline (previous)
- Result: Identification of new, updated, and deleted customers
- Null Handling: Nulls indicate new or deleted customers

### ### Aggregation Rules

#### 1. Customer Aggregate Spend:

- Group By: Name, Date
- Aggregation Function: sum(TotalAmount)

- Derived Column: TotalAmount = PricePerUnit Qty (calculated before aggregation)
- Output Columns: Name, TotalAmount, Date
- Null Handling: If PricePerUnit or Qty is NULL, TotalAmount will be NULL; sum() ignores NULLs

### ### SCD / History Tracking Rules

#### 1. SCD Type 2 Implementation:

- Tracking Columns:
- `IsActive` (Boolean): true for current version, false for historical versions
- `StartDate` (Timestamp): Effective start date of the record version
- `EndDate` (Timestamp): Effective end date of the record version (NULL for current)
- `OpTs` (Timestamp): Operation timestamp for versioning and precombine logic

#### 2. Change Detection:

- Compare all business columns (OrderId, CustId, ItemName, PricePerUnit, Qty, Date, Name, EmailId, Region)
- Exclude SCD metadata columns from comparison
- Any change in business columns triggers new version creation

#### 3. Version Management:

- New Record: Insert with IsActive=true, StartDate=current\_timestamp, EndDate=NULL, OpTs=current\_timestamp
- Changed Record:
- Update existing active record: Set IsActive=false, EndDate=current\_timestamp, OpTs=current\_timestamp
- Insert new active record: Set IsActive=true, StartDate=current\_timestamp, EndDate=NULL, OpTs=current\_timestamp
- Unchanged Record: No action required

#### 4. Hudi Configuration:

- Record Key: Composite of OrderId and CustId
- Precombine Key: OpTs (latest timestamp wins in case of conflicts)
- Table Type: COPY\_ON\_WRITE (optimized for read-heavy workloads)
- Operation: UPSERT (insert new records, update existing records)

### ### Null Handling

#### 1. Source Data:

- String literal "Null" is treated as invalid data and removed
- Actual NULL values are treated as invalid data and removed

- Both checks are case-sensitive for "Null" string

## 2. After Cleaning:

- No NULL values should exist in cleaned datasets
- Type casting may introduce NULLs if conversion fails (job should fail in this case)

## 3. Aggregation:

- NULL values in TotalAmount calculation are propagated (NULL anything = NULL)
- sum() function ignores NULL values in aggregation

## 4. SCD Type 2:

- EndDate is NULL for active records (by design)
- All other columns must not be NULL after cleaning

### ### Type Casting Expectations

#### 1. Order Data Type Casting:

- `PricePerUnit`: String → Decimal(10,2)
- `Qty`: String → Integer
- `Date`: String → Date
- Failure Handling: Job must fail if any casting fails

#### 2. Customer Data:

- All columns remain as String type (no casting required)

#### 3. SCD Metadata Columns:

- `IsActive`: Boolean (literal true/false)
- `StartDate`: Timestamp (from current\_timestamp())
- `EndDate`: Timestamp (NULL or from current\_timestamp())
- `OpTs`: Timestamp (from current\_timestamp())

#### 4. Aggregate Columns:

- `TotalAmount`: Decimal(18,2) (result of Decimal(10,2) Integer)
- 

### ## 6. Traceability Matrix

| Technical Source                         |
|------------------------------------------|
| -----                                    |
| s3://adif-sdlc/sdlc_wizard/customerdata/ |
| s3://adif-sdlc/sdlc_wizard/orderdata/    |

|               |                                                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------------------------------|
|               | df_customer_raw, df_order_raw                                                                                             |
|               | df_customer_null_removed, df_order_null_removed                                                                           |
|               | df_customer_clean                                                                                                         |
|               | df_order_clean                                                                                                            |
|               | df_customer_clean, df_order_clean, s3://adif-sdlc/curated/sdlc_wizard/ordersummary/                                       |
| ordersummary/ | df_customer_clean, s3://<BUCKET_NAME>/checkpoint/customer_baseline/, df_order_clean, s3://adif-sdlc/curated/sdlc_wizard/o |
|               | df_customer_clean, df_order_clean                                                                                         |

- Source Fidelity Note: All technical sources match the original sources specified in the FRD. No intermediate sources, staging tables, or alternative sources have been introduced. All transformations occur in-memory using Spark DataFrames with direct reads from original sources and direct writes to final targets.

- 

## ## 7. ETL Mapping Requirements

### ### TR-INGEST-001: Customer Data Ingestion Mapping

- Source: s3://adif-sdlc/sdlc\_wizard/customerdata/ (CSV)
- Source Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CustId      | String    | Variable               | Yes      |  |
|  | Name        | String    | Variable               | Yes      |  |
|  | EmailId     | String    | Variable               | Yes      |  |
|  | Region      | String    | Variable               | Yes      |  |

- Target: df\_customer\_raw (In-memory DataFrame)
- Target Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Mapping                        |  |
|--|-------------|-----------|------------------------|----------|--------------------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                          |  |
|  | CustId      | String    | Variable               | Yes      | Direct map from source.CustId  |  |
|  | Name        | String    | Variable               | Yes      | Direct map from source.Name    |  |
|  | EmailId     | String    | Variable               | Yes      | Direct map from source.EmailId |  |
|  | Region      | String    | Variable               | Yes      | Direct map from source.Region  |  |

- 

### ### TR-INGEST-002: Order Data Ingestion Mapping

- Source: s3://adif-sdlc/sdlc\_wizard/orderdata/ (CSV)



- Source Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable |  |
|--|--------------|-----------|------------------------|----------|--|
|  | -----        | -----     | -----                  | -----    |  |
|  | OrderId      | String    | Variable               | Yes      |  |
|  | ItemName     | String    | Variable               | Yes      |  |
|  | PricePerUnit | String    | Variable               | Yes      |  |
|  | Qty          | String    | Variable               | Yes      |  |
|  | Date         | String    | Variable               | Yes      |  |
|  | CustId       | String    | Variable               | Yes      |  |

- Target: df\_order\_raw (In-memory DataFrame)

- Target Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable | Mapping                             |  |
|--|--------------|-----------|------------------------|----------|-------------------------------------|--|
|  | -----        | -----     | -----                  | -----    | -----                               |  |
|  | OrderId      | String    | Variable               | Yes      | Direct map from source.OrderId      |  |
|  | ItemName     | String    | Variable               | Yes      | Direct map from source.ItemName     |  |
|  | PricePerUnit | String    | Variable               | Yes      | Direct map from source.PricePerUnit |  |
|  | Qty          | String    | Variable               | Yes      | Direct map from source.Qty          |  |
|  | Date         | String    | Variable               | Yes      | Direct map from source.Date         |  |
|  | CustId       | String    | Variable               | Yes      | Direct map from source.CustId       |  |

•

### ### TR-CLEAN-001: Null Removal Mapping

- Source: df\_customer\_raw, df\_order\_raw
- Target: df\_customer\_null\_removed, df\_order\_null\_removed
- Mapping: All columns are direct pass-through (no transformation). Rows containing NULL or "Null" string are filtered out.

•

### ### TR-CLEAN-002: Duplicate Removal Mapping

- Source: df\_customer\_null\_removed, df\_order\_null\_removed
- Target: df\_customer\_clean, df\_order\_clean
- Mapping: All columns are direct pass-through (no transformation). Duplicate rows are filtered out, retaining first occurrence.

•

### ### TR-CATALOG-001: Customer Catalog Registration Mapping

- Source: df\_customer\_clean

- Source Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CustId      | String    | Variable               | No       |  |
|  | Name        | String    | Variable               | No       |  |
|  | EmailId     | String    | Variable               | No       |  |
|  | Region      | String    | Variable               | No       |  |

- Target: gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_customer (Glue Catalog Table)

- Target Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Mapping                        |  |
|--|-------------|-----------|------------------------|----------|--------------------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                          |  |
|  | CustId      | String    | Variable               | No       | Direct map from source.CustId  |  |
|  | Name        | String    | Variable               | No       | Direct map from source.Name    |  |
|  | EmailId     | String    | Variable               | No       | Direct map from source.EmailId |  |
|  | Region      | String    | Variable               | No       | Direct map from source.Region  |  |

•

### ### TR-CATALOG-002: Order Catalog Registration Mapping

- Source: df\_order\_clean

- Source Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable |  |
|--|--------------|-----------|------------------------|----------|--|
|  | -----        | -----     | -----                  | -----    |  |
|  | OrderId      | String    | Variable               | No       |  |
|  | ItemName     | String    | Variable               | No       |  |
|  | PricePerUnit | String    | Variable               | No       |  |
|  | Qty          | String    | Variable               | No       |  |
|  | Date         | String    | Variable               | No       |  |
|  | CustId       | String    | Variable               | No       |  |

- Target: gen\_ai\_poc\_databrickscoe.sdmc\_wizard\_order (Glue Catalog Table)

- Target Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable | Mapping                             |  |
|--|--------------|-----------|------------------------|----------|-------------------------------------|--|
|  | -----        | -----     | -----                  | -----    | -----                               |  |
|  | OrderId      | String    | Variable               | No       | Direct map from source.OrderId      |  |
|  | ItemName     | String    | Variable               | No       | Direct map from source.ItemName     |  |
|  | PricePerUnit | String    | Variable               | No       | Direct map from source.PricePerUnit |  |
|  | Qty          | String    | Variable               | No       | Direct map from source.Qty          |  |
|  | Date         | String    | Variable               | No       | Direct map from source.Date         |  |
|  | CustId       | String    | Variable               | No       | Direct map from source.CustId       |  |

- 

### ### TR-SCD2-001: Order Summary SCD Type 2 Mapping

- Source 1: df\_customer\_clean

- Source 1 Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CustId      | String    | Variable               | No       |  |
|  | Name        | String    | Variable               | No       |  |
|  | EmailId     | String    | Variable               | No       |  |
|  | Region      | String    | Variable               | No       |  |

- Source 2: df\_order\_clean

- Source 2 Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable |  |
|--|--------------|-----------|------------------------|----------|--|
|  | -----        | -----     | -----                  | -----    |  |
|  | OrderId      | String    | Variable               | No       |  |
|  | ItemName     | String    | Variable               | No       |  |
|  | PricePerUnit | String    | Variable               | No       |  |
|  | Qty          | String    | Variable               | No       |  |
|  | Date         | String    | Variable               | No       |  |
|  | CustId       | String    | Variable               | No       |  |

- Target: gen\_ai\_poc\_databrickscoe.ordersummary (Hudi Table)

- Target Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Mapping                                          |  |
|--|-------------|-----------|------------------------|----------|--------------------------------------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                                            |  |
|  | OrderId     | String    | Variable               | No       | Direct map from df_order_clean.OrderId           |  |
|  | CustId      | String    | Variable               | No       | Direct map from df_order_clean.CustId (join key) |  |

|              |           |          |     |                                                               |  |
|--------------|-----------|----------|-----|---------------------------------------------------------------|--|
| ItemName     | String    | Variable | No  | Direct map from df_order_clean.ItemName                       |  |
| PricePerUnit | Decimal   | 10,2     | No  | Cast from df_order_clean.PricePerUnit (String → Decimal)      |  |
| Qty          | Integer   | -        | No  | Cast from df_order_clean.Qty (String → Integer)               |  |
| Date         | Date      | -        | No  | Cast from df_order_clean.Date (String → Date)                 |  |
| Name         | String    | Variable | No  | Direct map from df_customer_clean.Name (joined)               |  |
| EmailId      | String    | Variable | No  | Direct map from df_customer_clean.EmailId (joined)            |  |
| Region       | String    | Variable | No  | Direct map from df_customer_clean.Region (joined)             |  |
| IsActive     | Boolean   | -        | No  | Derived: true for current version, false for historical       |  |
| StartDate    | Timestamp | -        | No  | Derived: current_timestamp() when record becomes active       |  |
| EndDate      | Timestamp | -        | Yes | Derived: NULL for active, current_timestamp() when superseded |  |
| OpTs         | Timestamp | -        | No  | Derived: current_timestamp() for versioning                   |  |

•

### TR-INCR-001: Incremental Processing Mapping

- Source 1: df\_customer\_clean (current state)
- Source 2: s3:///checkpoint/customer\_baseline/ (previous state)
- Source 3: df\_order\_clean
- Source 4: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/ (existing Hudi table)
- Target 1: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/ (updated Hudi table)
- Target 2: s3:///checkpoint/customer\_baseline/ (new baseline)
- Mapping:
  - Change detection compares df\_customer\_clean with customer\_baseline on all columns
  - Changed customers are joined with df\_order\_clean to identify affected orders
  - SCD Type 2 logic is applied to affected records only (same mapping as TR-SCD2-001)
  - New baseline is direct copy of df\_customer\_clean
- 

### TR-AGG-001: Customer Aggregate Spend Mapping

- Source 1: df\_customer\_clean
- Source 1 Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable |  |
|--|-------------|-----------|------------------------|----------|--|
|  | -----       | -----     | -----                  | -----    |  |
|  | CustId      | String    | Variable               | No       |  |

|  |         |        |          |    |  |
|--|---------|--------|----------|----|--|
|  | Name    | String | Variable | No |  |
|  | EmailId | String | Variable | No |  |
|  | Region  | String | Variable | No |  |

- Source 2: df\_order\_clean

- Source 2 Schema:

|  | Column Name  | Data Type | Length/Precision/Scale | Nullable |  |
|--|--------------|-----------|------------------------|----------|--|
|  | -----        | -----     | -----                  | -----    |  |
|  | OrderId      | String    | Variable               | No       |  |
|  | ItemName     | String    | Variable               | No       |  |
|  | PricePerUnit | String    | Variable               | No       |  |
|  | Qty          | String    | Variable               | No       |  |
|  | Date         | String    | Variable               | No       |  |
|  | CustId       | String    | Variable               | No       |  |

- Target: gen\_ai\_poc\_databrickscoe.customeragggregatespend (Glue Catalog Table)

- Target Schema:

|  | Column Name | Data Type | Length/Precision/Scale | Nullable | Mapping                                                |  |
|--|-------------|-----------|------------------------|----------|--------------------------------------------------------|--|
|  | -----       | -----     | -----                  | -----    | -----                                                  |  |
|  | Name        | String    | Variable               | No       | Direct map from df_customer_clean.Name (joined)        |  |
|  | TotalAmount | Decimal   | 18,2                   | No       | Derived: sum(PricePerUnit * Qty) grouped by Name, Date |  |
|  | Date        | Date      | -                      | No       | Cast from df_order_clean.Date (String → Date)          |  |

- Derivation Details:
  - Join df\_order\_clean with df\_customer\_clean on CustId
  - Cast PricePerUnit to Decimal(10,2)
  - Cast Qty to Integer
  - Cast Date to Date
  - Calculate TotalAmount = PricePerUnit Qty for each order
  - Group by Name and Date
  - Aggregate: sum(TotalAmount) as TotalAmount
  - Select final columns: Name, TotalAmount, Date
  - 
  - End of Technical Requirements Document